

SharedNAS — USB

Readme

The QRP Labs QMX/QMX+ that is shared from the Synology NAS, presented on a Mac as a serial port, so CAT software that expects a USB-connected QMX can use it over the network

Item	Value
Program	tools/mac-serial-bridge/qmx_serial_bridge.py (Python 3, standard library only)
Project folder	~/src/qmx-panadapter-v1.16/tools/mac-serial-bridge (work tree, branch network-radio); in every NAS build as kit patch 0010, e.g. ~/src/QMX-Panadapter-NAS/tools/mac-serial-bridge
Finds the NAS	Broadcasts QMXR? on UDP 7355; the relay answers QMXR! <ip> 7355 (e.g. QMXR! 10.0.0.137 7355) - same search as the Tab5's Find NAS
Connects to	The NAS relay (tools/nas-relay/qmx_nas_relay.py), QMX serial port 3 on TCP 7356, at the address the NAS answered with
Presents	A serial port on the Mac at ~/cu.QMX-NAS (a pseudo-terminal)
Carries	CAT / serial data only (no audio)
Runs on	macOS (installed as a login item, LaunchAgent); also runs on Linux
Relay or firmware changes	None - uses the relay's existing port 3 service
Document date	October 08, 2026 (adds NAS discovery)

Contents

- 1. Summary of outcome
- 2. Background and findings
- 3. Architecture
- 4. How the bridge works and finds the NAS
- 5. Verification performed
- 6. Deployment steps
- 7. Using it from applications
- 8. Known limitations and open items
- 9. Sources (references)
- 10. Driver listings (serial port bridge)

- 11. Application listings (install, uninstall, readme, test tools)

1. Summary of outcome

A new program, **qmx_serial_bridge.py**, makes the QMX that is plugged into the Synology NAS appear on a Mac as a serial port. Programs that normally talk to a USB-connected QMX (WSJT-X, JTDX, fldigi, Hamlib/rigctld, logging programs, a terminal) open `~/cu.QMX-NAS` and control the radio over the network exactly as they would over a USB cable.

- It connects to the relay's existing **port 3** service (TCP 7356), which serves the QMX's third USB serial port as raw bytes. Nothing changes on the NAS or the Tab5, and the Tab5 and the Mac can control the radio at the same time, each on its own serial port.
- It needs only the Python 3 that comes with macOS (Xcode command line tools). **install-mac.sh** installs it as a login item that starts automatically and restarts if it stops, then runs a read-only check against the radio.
- It finds the NAS by itself: it broadcasts **QMXR?** on UDP 7355, the same search the Tab5's **Find NAS** uses, and the relay answers straight back with **QMXR! 10.0.0.137 7355**. The bridge takes the address from that answer and searches again if the NAS stops answering there.
- It connects to the NAS only while a program is using the port, and releases the radio's port 3 after 120 seconds of silence.
- Tested on a Linux bench against the current relay code in fake-radio mode (discovery, probe and the relay's own nettest all pass) and against stand-ins: CAT round trips, idle release and reconnect, relay down, port 3 refused, echo turned on by an application, pyserial, several relays and junk answers, and the NAS changing address while the bridge runs. **Not yet run on a Mac or against the real NAS and radio** (see section 5).

2. Background and findings

Starting point

- The NAS relay (tools/nas-relay) already serves two things: the QMXR/1 stream for the Tab5 on TCP 7355, and - with **System config → GPS & Ser. ports → USB serial ports = 3** on the radio - the QMX's third serial port as a plain TCP serial port on **7356**, passed through unchanged in both directions.
- The relay now also answers a UDP **QMXR?** on port 7355 with **QMXR! <its IP address> <TCP port>**, picking the address of the interface that reaches the asker (commit b62aa45, 'Find NAS'). The Tab5 sends it three times over 1.5 s and collects distinct answers.
- The relay README suggested mapping 7356 to a serial port with socat. The goal here was a dedicated program that does this reliably on the Mac, starts by itself, and handles the radio or NAS going away.

Relay behaviour the bridge relies on (read from qmx_nas_relay.py)

Relay behaviour	Effect on the bridge
SerialClient opens QMX port 3 when a client connects and releases it when the client leaves.	Connecting on demand and releasing when idle means port 3 is only held while in use.
If the radio has fewer than 3 serial ports, the connection is closed straight away.	A close (or reset) within 1.5 s of connecting, with no reply, is reported as 'set USB serial ports to 3'.
One port 3 client at a time; a new connection replaces the old one.	One Mac at a time; a second Mac takes over from the first.
If the QMX is powered off or unplugged, the relay closes the connection.	The bridge releases the relay and reconnects on the next command.
Bytes pass through with no framing, like a serial cable.	The bridge copies bytes unchanged in both directions.

Relay behaviour	Effect on the bridge
Discovery answer is QMXR! <ip> <port>, where the port is the QMXR/1 Tab5 service (7355).	The bridge uses the answered IP address with the port 3 service, TCP 7356 (--port). 7355 carries the Tab5's framed I/Q stream and only one Tab5 session, so it can't be used as a raw serial port.

Chosen approach

A pseudo-terminal (pty) is the same kind of character device that programs open for a USB serial adapter, and macOS lets any user create one without drivers or administrator rights. The bridge creates a pty, links it at a fixed path, and copies bytes between it and the relay's TCP port 3. A device that macOS itself lists as a USB serial port would need a DriverKit system extension, which requires driver entitlements granted by Apple and a signed, notarized app. That was not practical here (see section 8).

3. Architecture

```

QMX/QMX+ --USB--> Synology NAS 10.0.0.137 (qmx_nas_relay.py)
|
|-- UDP 7355  QMXR? <-- broadcast by the Mac bridge (and the Tab5)
|              QMXR! 10.0.0.137 7355 --> straight back to the asker
|
|-- TCP 7355  QMXR/1: I/Q + CAT + Radio menus --> Tab5 QMX-Panadapter
|
|-- TCP 7356  QMX serial port 3, raw bytes
|
| v
Mac: qmx_serial_bridge.py (LaunchAgent com.qmx.serial-bridge)
|
| v
pseudo-terminal /dev/ttysNNN <-- symlink ~/cu.QMX-NAS
|
| ^
|
WSJT-X / JTDX / fldigi / rigctld / logger / screen

```

Part	Where	Role
qmx_nas_relay.py	NAS (unchanged by this work)	Owns the QMX over USB; serves port 3 on TCP 7356; answers QMXR? on UDP 7355
qmx_serial_bridge.py	Mac, ~/Library/Application Support/QMX-Serial-Bridge	Creates the serial port and bridges it to TCP 7356
com.qmx.serial-bridge.plist	Mac, ~/Library/LaunchAgents	Starts the bridge at login and restarts it if it stops
~/cu.QMX-NAS	Mac, home folder	Fixed path that always points at the current pty
qmx-serial-bridge.log	Mac, ~/Library/Logs	Connections, releases, warnings

4. How the bridge works and finds the NAS

Behaviour	Detail
Finding the NAS	Unless --host is given, the bridge sends QMXR? to UDP 7355 three times over 1.5 s, to 255.255.255.255 and to each network interface's own broadcast address (read from ifconfig), so the NAS's network is asked even with Wi-Fi and Ethernet both up. Answers starting QMXR! are collected; duplicates and anything else are ignored.
Using the answer	The address in QMXR! <ip> <port> is used (or the packet's source address if it doesn't parse). The bridge connects to that address on TCP 7356, the port 3 service. If several relays answer, the first is used and the rest are logged.
When it searches	At start-up, and again whenever the remembered address refuses or times out, at most every 3 s. If the NAS gets a new address, the next command follows it.

Behaviour	Detail
No answer	With --fallback-host (the installer sets 10.0.0.137) that address is used; otherwise commands get no reply and the search is repeated. --host fixes the address and never searches. --discover lists the relays that answer and exits.
Serial port	os.openpty() creates the pty; it is set to raw 8-bit with echo off, and a symlink (created atomically) points ~/cu.QMX-NAS at it. The bridge keeps its own handle on the device open, so the device stays in place while programs open and close it.
Connect on demand	Nothing connects to the NAS until a program writes to the port. The first bytes trigger the connection (3 s timeout) and are then sent.
Idle release	After --idle seconds (default 120) with no traffic either way, the connection is closed and the radio's port 3 is released. --idle 0 never releases; --keep-connected connects at start and reconnects all the time.
Radio unavailable	If the NAS can't be reached, commands are dropped (no reply, like a radio that is off) and connecting is retried after 3 s. The log notes it at most every 30 s.
Port 3 refused	A close or reset within 1.5 s of connecting, with no reply, is logged as: is the QMX on, and is 'USB serial ports' set to 3?
Echo guard	Before every write to the port the bridge checks the port settings and turns echo off if a program turned it on. Echo would send the radio's replies back into the radio.
Stale data	Radio bytes nobody has read are flushed when a new connection starts and on idle release, so the next program never sees an old reply.
Port full	Writes to the port are non-blocking. If no program is reading and the buffer is full, the bytes are dropped and logged rather than stalling the bridge.
Shutdown	SIGTERM, SIGINT or SIGHUP closes the connection and removes the symlink if it still points at this bridge's pty.
--probe	One-off read-only check: finds the NAS (unless --host), sends ID; FA; MD; to port 3 and prints PROBE PASSED or FAILED. Nothing that transmits.
-v	Logs every byte in both directions.

5. Verification performed

Bench tests on Linux (Python 3.13) with test/fake_port3_relay.py standing in for the relay's port 3 and test/port_client.py or pyserial acting as the application. The bridge ran with --idle 3.

Test	Result
Relay down: application sends FA;	No reply; log: can't reach the relay, dropping commands (retrying). Pass
Relay up after the retry window: ID; FA; FA00007074000;FA; MD;	ID020; FA00014074000; FA00007074000; MD3; - set and read back correctly. Pass
Application turns echo on, then sends FA;	Echo turned off by the bridge; the relay received only FA; (no echoed reply). Pass
Idle release (3 s), then FA;	Released after 3 s, reconnected on the next command, correct reply. Pass
Relay killed while the bridge is connected	Connection released; FA; gets no reply. Pass
Radio without port 3 (relay closes at once)	Log: relay closed port 3 straight away ... 'USB serial ports' set to 3? Pass
pyserial 3.5 opens the port at 38400 baud, sends ID; FA;	ID020; FA00014074000; Pass
Bridge stopped (SIGTERM)	~/cu.QMX-NAS symlink removed. Pass
--probe against the stand-in	ok ID; ok FA; ok MD; PROBE PASSED. Pass

Test	Result
Current relay (qmx_nas_relay.py --fake, commit b62aa45): --discover	QMXR! 192.0.2.2 7355 -> serial port 3 at 192.0.2.2:7356 (the test machine's address). Pass
Current relay: --probe with no address	QMXR? answered, connected to port 3, ID020; FA00014074000; MD2; PROBE PASSED. Pass
Current relay: the relay's own nettest.py	NETTEST PASSED (Tab5 and PC on port 3 together). Pass
Current relay: bridge started with no address, application sends ID; FA; MD;	Log: found the NAS relay: QMXR! 192.0.2.2 7355 - using 192.0.2.2:7356; correct replies. Pass
No relay answering	--discover: no relay answered (exit 1); --probe FAIL; with --fallback-host the fallback address is tried. Pass
Answers: junk, 'QMXR? x', two relays, a duplicate, 'QMXR! notanip 7355'	Junk ignored, duplicate merged, both relays listed, bad address replaced by the packet's source. Pass
NAS moves from 127.0.0.2 to 127.0.0.3 while the bridge runs	Old address refused; search found 127.0.0.3; the same command was answered by the new NAS. Pass
install-mac.sh dry run (launchctl and xcode-select stubbed)	LaunchAgent arguments: search with --fallback-host 10.0.0.137; with an IP: --host IP; EXTRA added. Pass. Not run on macOS.
--probe against the real NAS 10.0.0.137	Not run: the test machine could not reach the home network (network unreachable).

6. Deployment steps

- On the radio: **System config → GPS & Ser. ports → USB serial ports = 3.**
- Make sure the relay log on the NAS shows listening on 0.0.0.0:7356.
- On the Mac: `cd ~/src/QMX-Panadapter-NAS/tools/mac-serial-bridge` then `bash install-mac.sh` (or `bash install-mac.sh <NAS IP>`). For an always-held connection: `EXTRA="--keep-connected"` `bash install-mac.sh`.
- The installer copies the program, writes the LaunchAgent, starts it, shows the last log lines and runs the probe, which should end with **PROBE PASSED**.
- The bridge finds the NAS with QMXR? by itself; the installer adds `--fallback-host 10.0.0.137` in case broadcasts are blocked. `bash install-mac.sh <NAS IP>` fixes the address instead.
- If DSM's firewall is on, allow UDP 7355 as well as TCP 7356. The Mac and the NAS must be on the same network for the search to reach it.
- To list the relays that answer: `python3 qmx_serial_bridge.py --discover`. To run it by hand: `python3 qmx_serial_bridge.py -v`.
- To remove it: `bash uninstall-mac.sh` (the log is kept).
- If macOS asks to install the command line developer tools when Python 3 runs, accept; the installer checks for them first.

7. Using it from applications

Point the program at the serial port by typing its full path, `/Users/jpwatters/cu.QMX-NAS`. Any baud rate works; the setting is ignored.

Application	Rig	Port / settings
WSJT-X, JTDX	QRP Labs QMX if listed, otherwise Kenwood TS-480	Serial port: type the path. PTT method: CAT.
fldigi (Hamlib)	Same as above	Device: type the path.
Hamlib rigctld	-m with the QMX or TS-480 model number	<code>rigctld -m <model> -r ~/cu.QMX-NAS</code>
Terminal check	-	<code>screen ~/cu.QMX-NAS 38400</code> then type FA; (the radio answers with its frequency)

PTT must be by CAT: DTR and RTS lines are not carried over the network.

8. Known limitations and open items

- **Serial (CAT) only, no audio.** Digital modes still need the radio's audio on the Mac by another route.
- **Not a USB device in macOS's device list.** The port is a pseudo-terminal at a typed path. Programs whose port list cannot be edited will not show it. A true USB serial device would need a DriverKit system extension with Apple-granted driver entitlements.
- One PC at a time on port 3; a second Mac connecting takes over from the first.
- The first command after an idle release waits for the connection (normally well under a second); a program with a very short timeout may miss that one reply and succeed on its retry.
- The relay doesn't advertise the port 3 service in its QMXR! answer, so the bridge assumes TCP 7356 (`--port`). If the relay's `--serial-port` is ever changed, set `--port` to match.
- Broadcasts stay on the local network: a Mac on another subnet or VPN needs `--host`.
- macOS can block local-network access for python3 (System Settings → Privacy & Security → Local Network); then both the search and the connection fail, and the log points there.
- **Open:** run `install-mac.sh` on the Mac, confirm the probe finds 10.0.0.137 and PASSES, and try WSJT-X on `~/cu.QMX-NAS` with the Tab5 connected at the same time.
- Committed on branch `network-radio` in `~/src/qmx-panadapter-v1.16` and added to the NAS kit as patch 0010, so every NAS build made from a new upstream release includes the bridge.

9. Sources (references)

Each source is kept whole on one page. “Verified” means it was read or executed during this work; “background knowledge” means it was relied on but not re-checked here.

[1] QMX NAS relay readme

Location: `~/src/QMX-Panadapter-NAS/tools/nas-relay/README.md`

Used for: Port layout (port 1 CAT for the Tab5, port 2 Radio menus, port 3 for PC software on TCP 7356), the 'USB serial ports = 3' radio setting, one PC at a time, port 3 closed at once when the radio has 1 or 2 ports, serial data only (no audio). Verified.

[2] QMX NAS relay program

Location: `~/src/QMX-Panadapter-NAS/tools/nas-relay/qmx_nas_relay.py` (SerialClient, serve(), DEFAULT_SERIAL_PORT)

Used for: Port 3 behaviour: raw pass-through, QMX port opened on connect and released on disconnect, refusal closes the socket, new connection replaces the old one, TCP_NODELAY and keepalive. Verified.

[3] Relay network test

Location: `~/src/QMX-Panadapter-NAS/tools/nas-relay/nettest.py`

Used for: Pattern for a read-only port 3 check (send FA;, read to ';'), used for the --probe mode. Verified.

[4] Relay discovery responder

Location: `~/src/QMX-Panadapter-NAS/tools/nas-relay/qmx_nas_relay.py` (discovery_responder, local_ip_for, DISCOVERY_QUERY/REPLY, --discovery-port)

Used for: UDP 7355 QMXR? query, QMXR! <ip> <tcp port> answer sent straight back to the asker, the answered port being the QMXR/1 service. Verified; run with --fake in the tests.

[5] Tab5 discovery client

Location: `~/src/QMX-Panadapter-NAS/main/net/radio_relay.c` (radio_relay_discover) and `radio_relay.h`

Used for: Search pattern copied by the bridge: three broadcasts over the timeout, answers starting 'QMXR!', source address used when the answer's address doesn't parse, duplicates merged. Verified (read).

[6] Project repository history

Location: `~/src/QMX-Panadapter-NAS` (git describe v1.16.12-9-gb62aa45; b62aa45 'network radio: Find NAS - the Tab5 asks the network for the relay's address')

Used for: Confirms discovery and the port 3 service are in the current NAS build. Verified.

[7] NAS kit

Location: `~/src/qmx-panadapter-v1.16/tools/nas-kit` (README.md, make-qmx-panadapter-nas.sh, patches/)

Used for: How NAS builds are made from each upstream release by applying the patch series, and how a new NAS commit is exported into it; the bridge is added there as patch 0010. Verified (read).

[8] Python standard library: os, Pty, TTY, Termios, select, socket

Location: <https://docs.python.org/3/library/termios.html> ;
<https://docs.python.org/3/library/os.html#os.openpty>

Used for: openpty, ttyname, tcgetattr/tcsetattr (ECHO, ECHONL), tcflush(TCIFLUSH), tty.setraw, non-blocking I/O, UDP broadcast (SO_BROADCAST). Background knowledge; behaviour exercised in the bench tests on Linux.

[9] pySerial 3.5

Location: <https://pypi.org/project/pyserial/>

Used for: Used in the bench test as a real application library opening the port at 38400 baud. Verified (executed).

[10] Apple launchd and LaunchAgents

Location: `launchd.plist(5)` and `launchctl(1)` manual pages on macOS

Used for: `RunAtLoad`, `KeepAlive`, `ThrottleInterval`, `StandardOutPath`; `launchctl bootstrap/bootout` gui/. Background knowledge; not run on macOS in this work.

[11] Apple DriverKit / USBSerialDriverKit

Location: <https://developer.apple.com/documentation/driverkit>

Used for: Basis for the limitation that a device macOS lists as a USB serial port needs a system extension with Apple-granted entitlements. Background knowledge.

[12] Hamlib

Location: <https://hamlib.github.io/>

Used for: Rig control used by WSJT-X, JTDX, fldigi and rigctld; accepts a serial device path typed by the user. Background knowledge.

[13] QRP Labs QMX / QMX+ operating manual

Location: <https://qrp-labs.com/qmx>

Used for: System config → GPS & Ser. ports → USB serial ports; Kenwood TS-480-style CAT commands (ID; FA; MD;). Referenced through the relay readme. Background knowledge.

[14] Upstream firmware: SteffenLav/qmx-panadapter

Location: <https://github.com/SteffenLav/qmx-panadapter> (v1.16.12, network-radio work in `~/src/QMX-Panadapter-NAS`)

Used for: Project this bridge belongs to; version label v1.16.12-NAS. Background.

10. Driver listings (serial port bridge)

The program that presents the NAS-shared QMX as a serial port on the Mac, listed in full from tools/mac-serial-bridge (in ~/src/qmx-panadapter-v1.16 and every NAS build). Long lines are wrapped (↵).

tools/mac-serial-bridge/qmx_serial_bridge.py (new)

```
#!/usr/bin/env python3
"""
qmx_serial_bridge.py - make the QMX on the Synology NAS show up on a Mac as a
serial port, so WSJT-X, Hamlib/rigctld, fldigi, loggers or a terminal can use
it as if the radio were plugged into the Mac's USB.

    QMX --USB--> NAS (qmx_nas_relay.py) --TCP 7356 (QMX serial port 3)-->
    this program --> serial device on the Mac (~/.cu.QMX-NAS by default)

Finding the NAS: unless --host is given, it asks the network the same way the
Tab5's Find NAS does - broadcasts "QMXR?" on UDP 7355 and the relay answers
"QMXR! <its IP address> <TCP port>". The address is taken from that answer.
The port in the answer is the Tab5's QMXR/1 service (7355); this program uses
the relay's port 3 service on the same NAS, TCP 7356 (--port).
If the NAS stops answering at the remembered address, it searches again.

It creates a pseudo-terminal (the same kind of device a USB serial adapter
gives you) and links it at a fixed path. Point the program at that path the
same way you would for a USB-connected QMX (any baud rate works; the setting
is ignored).

* Connects to the relay only when a program actually talks to the port, and
  lets go after --idle seconds of silence, so the radio's port 3 is only
  held while something is using it (--keep-connected holds it all the time).
* If the QMX is off or the relay is down, what the program sends is dropped
  and it retries on the next command, like a radio that is switched off.
* Bytes pass through unchanged in both directions - no framing.
* If the NAS is not found it falls back to --fallback-host, if given.

Needs only the Python 3 that comes with macOS (Xcode command line tools).
Also runs on Linux.

    python3 qmx_serial_bridge.py                # find the NAS, ~/.cu.QMX-NAS
    python3 qmx_serial_bridge.py --discover      # list the relays that answer, then exit
    python3 qmx_serial_bridge.py --host 10.0.0.137 # fixed address, no search
    python3 qmx_serial_bridge.py --probe        # one-off check: ID; FA; MD;
"""

from __future__ import annotations

import argparse
import errno
import logging
import os
import select
import signal
import re
import socket
import subprocess
import sys
import termios
import time
import tty

log = logging.getLogger("qmx-serial-bridge")

DEFAULT_PORT = 7356          # relay: QMX serial port 3, raw bytes
DISCOVERY_PORT = 7355        # relay: answers "QMXR?" on UDP (same number as its QMXR/1 TCP port)
DISCOVERY_TIME = 1.5         # seconds; "QMXR?" is sent 3 times in this window, like the Tab5
DISCOVERY_QUERY = b"QMXR?"
DISCOVERY_REPLY = b"QMXR!"
DEFAULT_LINK = "~/cu.QMX-NAS"
DEFAULT_IDLE = 120           # seconds without traffic before the relay port is released
CONNECT_TIMEOUT = 3.0
RETRY_AFTER = 3.0            # after a failed connect, drop commands for this long
QUICK_CLOSE = 1.5           # relay closing this fast after connect = port 3 refused

def fmt(b: bytes, n: int = 60) -> str:
    s = b.decode("ascii", "replace")
    return s if len(s) <= n else s[:n] + "..."
def broadcast_addresses() -> list[str]:
    """255.255.255.255 plus each interface's own broadcast address. A Mac sends
```

```

255.255.255.255 out of one interface only, so with Wi-Fi and Ethernet both
up the directed addresses make sure the NAS's network is asked too.""
out = ["255.255.255.255"]
try:
    txt = subprocess.run(["ifconfig"], capture_output=True, text=True, timeout=3).stdout
    for b in re.findall(r"\bbroadcast (\d+\.\d+\.\d+\.\d+)", txt) + \
        re.findall(r"\bBcast:(\d+\.\d+\.\d+\.\d+)", txt):
        if b not in out:
            out.append(b)
except (OSError, subprocess.SubprocessError):
    pass
return out

def discover(timeout: float = DISCOVERY_TIME, udp_port: int = DISCOVERY_PORT,
            targets: list[str] | None = None) -> list[tuple[str, int]]:
    """Ask the network for QMX relays: send "QMXR?" three times over `timeout`
    seconds and collect the distinct "QMXR! <ip> <tcp port>" answers, in the
    order they arrive. The answer's own address is used; if it doesn't parse,
    the address the answer came from. Returns [(ip, advertised tcp port), ...]."""
    targets = targets or broadcast_addresses()
    found: list[tuple[str, int]] = []
    u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        u.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
        u.settimeout(0.1)
        start = time.monotonic(), 0
        while time.monotonic() - start < timeout:
            if sent < 3 and time.monotonic() - start >= sent * timeout / 3:
                for t in targets:
                    try:
                        u.sendto(DISCOVERY_QUERY, (t, udp_port))
                    except OSError as e:
                        log.debug("discovery: send to %s failed: %s", t, e)
                sent += 1
            try:
                data, peer = u.recvfrom(256)
            except socket.timeout:
                continue
            except OSError as e:
                log.debug("discovery: %s", e)
                continue
            parts = data.decode("ascii", "replace").split()
            if not parts or parts[0] != DISCOVERY_REPLY.decode():
                continue
            ip = parts[1] if len(parts) > 1 else ""
            try:
                socket.inet_aton(ip)
                if ip.count(".") != 3:
                    raise OSError
            except OSError:
                ip = peer[0]
            port = int(parts[2]) if len(parts) > 2 and parts[2].isdigit() else DISCOVERY_PORT
            if not 0 < port < 65536:
                port = DISCOVERY_PORT
            if (ip, port) not in found:
                found.append((ip, port))
    finally:
        u.close()
    return found

class Bridge:
    def __init__(self, args):
        self.args = args
        self.link = os.path.expanduser(args.link)
        self.master = self.slave = -1
        self.sock: socket.socket | None = None
        self.connected_at = 0.0
        self.got_reply = False
        self.last_traffic = 0.0
        self.retry_at = 0.0
        self.dropped_note = 0.0
        self.running = True
        self.host = args.host # None = find it with "QMXR?"
        self.searched_at = -1e9

    # ---- finding the NAS -----

    def find_nas(self) -> bool:
        """Ask the network where the relay is (not more than every RETRY_AFTER s)."""
        now = time.monotonic()

```

```

if now - self.searched_at < RETRY_AFTER:
    return self.host is not None
self.searched_at = now
found = discover(udp_port=self.args.discovery_port)
if found:
    ip, adv = found[0]
    if ip != self.host:
        log.info("found the NAS relay: QMXR! %s %d - using %s:%d for QMX port 3",
                 ip, adv, ip, self.args.port)
    if len(found) > 1:
        log.info("other relays answered too (using the first): %s",
                 ", ".join("%s:%d" % f for f in found[1:]))
    self.host = ip
    return True
if self.args.fallback_host:
    if self.host != self.args.fallback_host:
        log.warning("no relay answered QMXR? on UDP %d; using --fallback-host %s",
                    self.args.discovery_port, self.args.fallback_host)
    self.host = self.args.fallback_host
    return True
log.warning("no relay answered QMXR? on UDP %d (is the relay running, and the Mac "
            "on the same network as the NAS?)", self.args.discovery_port)
return self.host is not None

# ---- the serial device -----

def open_pty(self):
    self.master, self.slave = os.openpty()
    # Raw 8-bit, no echo: an echo would send the radio's replies back to it.
    tty.setraw(self.slave)
    self.no_echo()
    os.set_blocking(self.master, False)
    name = os.ttyname(self.slave)
    # We keep our own handle on the device open (and never read it), so the
    # device stays put while programs open and close it.
    d = os.path.dirname(self.link)
    if d:
        os.makedirs(d, exist_ok=True)
    tmp = "%s.%d.tmp" % (self.link, os.getpid())
    os.symlink(name, tmp)
    os.replace(tmp, self.link)
    log.info("serial port ready: %s (device %s)", self.link, name)

def no_echo(self):
    """Programs may change the port settings; never allow echo, which would
    loop the radio's replies straight back into the radio."""
    try:
        a = termios.tcgetattr(self.slave)
        bad = termios.ECHO | termios.ECHONL
        if a[3] & bad:
            a[3] &= ~bad
            termios.tcsetattr(self.slave, termios.TCSANOW, a)
            log.info("turned echo off on the serial port")
    except termios.error:
        pass

def flush_stale(self):
    """Throw away radio bytes no program has read (e.g. a reply that came
    after the program closed the port), so the next program doesn't see them."""
    try:
        termios.tcflush(self.slave, termios.TCIFLUSH)
    except termios.error:
        pass

def to_port(self, data: bytes):
    self.no_echo()
    view = memoryview(data)
    while view:
        try:
            n = os.write(self.master, view)
            view = view[n:]
        except BlockingIOError:
            log.warning("serial port full (nothing reading it): dropped %d bytes", len(view))
            return
        except OSError as e:
            log.warning("serial port write failed: %s", e)
            return

# ---- the relay -----

def connect(self) -> bool:
    a = self.args

```

```

searching = a.host is None
if self.host is None and not (searching and self.find_nas()):
    self.retry_at = time.monotonic() + RETRY_AFTER
    return False
try:
    s = socket.create_connection((self.host, a.port), CONNECT_TIMEOUT)
except OSError as e:
    log.warning("can't reach the relay at %s:%d: %s", self.host, a.port, e)
    if e.errno in (errno.EHOSTUNREACH, errno.ENETUNREACH, errno.EPERM):
        log.warning("if this Mac is on the NAS's network, check System Settings -> "
                    "Privacy & Security -> Local Network allows python3")
    old = self.host
    if searching and self.find_nas() and self.host != old:
        return self.connect() # the NAS moved: try its new address once
    self.retry_at = time.monotonic() + RETRY_AFTER
    return False
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
s.setblocking(False)
self.sock = s
self.connected_at = self.last_traffic = time.monotonic()
self.got_reply = False
self.flush_stale()
log.info("connected to the relay %s:%d (QMX serial port 3)", self.host, a.port)
return True

def disconnect(self, why: str):
    if self.sock:
        try:
            self.sock.close()
        except OSError:
            pass
        self.sock = None
        log.info("released the relay: %s", why)

def to_relay(self, data: bytes):
    try:
        self.sock.setblocking(True)
        self.sock.settimeout(CONNECT_TIMEOUT)
        self.sock.sendall(data)
    except OSError as e:
        self.disconnect("send failed: %s" % e)
    finally:
        if self.sock:
            self.sock.setblocking(False)

# ---- main loop -----

def on_port_data(self):
    try:
        data = os.read(self.master, 4096)
    except BlockingIOError:
        return
    except OSError as e:
        if e.errno == errno.EIO: # no program has the port open
            time.sleep(0.05)
            return
        raise
    if not data:
        return
    log.debug("port -> radio: %s", fmt(data))
    now = time.monotonic()
    if not self.sock:
        if now < self.retry_at or not self.connect():
            if now - self.dropped_note > 30:
                log.info("radio not reachable: dropping commands (retrying)")
                self.dropped_note = now
            return
        self.last_traffic = now
        self.to_relay(data)

def on_relay_data(self):
    try:
        data = self.sock.recv(4096)
    except BlockingIOError:
        return
    except OSError as e:
        if not self.quick_close():
            self.disconnect("connection lost: %s" % e)
        return
    if not data:
        if not self.quick_close():

```

```

        self.disconnect("relay closed the connection (QMX off or relay restarted)")
    return
    self.got_reply = True
    self.last_traffic = time.monotonic()
    log.debug("radio -> port: %s", fmt(data))
    self.to_port(data)

def quick_close(self) -> bool:
    """The relay drops port 3 at once when the radio can't serve it."""
    if not self.got_reply and time.monotonic() - self.connected_at < QUICK_CLOSE:
        log.warning("the relay closed port 3 straight away. Is the QMX on, and is "
                    "'USB serial ports' set to 3 on the radio? (see the relay's log)")
        self.retry_at = time.monotonic() + RETRY_AFTER
        self.disconnect("port 3 refused")
        return True
    return False

def run(self):
    self.open_ptty()
    a = self.args
    if a.host is None:
        self.find_nas()
    log.info("relay: %s:%d %s", self.host or "(not found yet)", a.port,
            "kept connected" if a.keep_connected else
            "connects on demand, releases after %ds idle" % a.idle if a.idle else
            "connects on demand, stays connected")
    try:
        while self.running:
            now = time.monotonic()
            if a.keep_connected and not self.sock and now >= self.retry_at:
                self.connect()
            if (self.sock and a.idle and not a.keep_connected
                and now - self.last_traffic > a.idle):
                self.disconnect("idle for %ds" % a.idle)
                self.flush_stale()
            rl = [self.master] + ([self.sock] if self.sock else [])
            try:
                r, _, _ = select.select(rl, [], [], 1.0)
            except InterruptedError:
                continue
            if self.sock and self.sock in r:
                self.on_relay_data()
            if self.master in r:
                self.on_port_data()
        finally:
            self.cleanup()

def cleanup(self):
    self.disconnect("stopping")
    try:
        if os.path.islink(self.link) and self.slave >= 0 and \
            os.readlink(self.link) == os.ttyname(self.slave):
            os.unlink(self.link)
    except OSError:
        pass
    for fd in (self.slave, self.master):
        if fd >= 0:
            try:
                os.close(fd)
            except OSError:
                pass

def probe(args) -> int:
    """One-off check through the relay: read-only CAT queries, never transmits."""
    host = args.host
    if host is None:
        found = discover(udp_port=args.discovery_port)
        if found:
            print("ok : QMXR? -> %s" % " ".join("QMXR! %s %d" % f for f in found))
            host = found[0][0]
        elif args.fallback_host:
            print("note: no relay answered QMXR?; using --fallback-host %s" % args.fallback_host)
            host = args.fallback_host
        else:
            print("FAIL: no relay answered QMXR? on UDP %d" % args.discovery_port)
            return 1
    try:
        s = socket.create_connection((host, args.port), CONNECT_TIMEOUT)
    except OSError as e:
        print("FAIL: can't reach the relay at %s:%d: %s" % (host, args.port, e))
        return 1

```

```

print("ok : connected to %s:%d (QMX port 3)" % (host, args.port))
s.settimeout(3)
ok = True
for q in (b"ID;", b"FA;", b"MD;"):
    s.sendall(q)
    reply, end = b"", time.monotonic() + 3
    while time.monotonic() < end and not reply.endswith(b";"):
        try:
            c = s.recv(256)
        except socket.timeout:
            break
        if not c:
            print("FAIL: the relay closed port 3 - is the QMX on, with "
                  "'USB serial ports' = 3?")
            return 1
        reply += c
    good = reply.endswith(b";")
    ok &= good
    print("%s: %s -> %s" % ("ok " if good else "FAIL", q.decode(), fmt(reply) or "(no reply)"))
s.close()
print("PROBE %s" % ("PASSED" if ok else "FAILED"))
return 0 if ok else 1

def main():
    p = argparse.ArgumentParser(description="QMX on the NAS as a serial port on this Mac")
    p.add_argument("--host", default=os.environ.get("QMX_NAS_HOST") or None,
                   help="fixed NAS address; without it the NAS is found by asking QMXR?")
    p.add_argument("--fallback-host", default=None,
                   help="address to use when no relay answers the search")
    p.add_argument("--discovery-port", type=int, default=DISCOVERY_PORT,
                   help="UDP port the relay answers QMXR? on (default %(default)s)")
    p.add_argument("--discover", action="store_true",
                   help="list the relays that answer QMXR? and exit")
    p.add_argument("--port", type=int, default=DEFAULT_PORT,
                   help="relay port for QMX serial port 3 (default %(default)s)")
    p.add_argument("--link", default=DEFAULT_LINK,
                   help="path of the serial port to create (default %(default)s)")
    p.add_argument("--idle", type=int, default=DEFAULT_IDLE,
                   help="release the relay after this many idle seconds, 0 = never (default %(default)s)")
    p.add_argument("--keep-connected", action="store_true",
                   help="stay connected to the relay all the time")
    p.add_argument("--probe", action="store_true",
                   help="check the relay once (ID; FA; MD;) and exit")
    p.add_argument("-v", "--verbose", action="store_true", help="log every byte")
    args = p.parse_args()
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s", stream=sys.stdout)

    if args.discover:
        found = discover(udp_port=args.discovery_port)
        for ip, port in found:
            print("QMXR! %s %d -> serial port 3 at %s:%d" % (ip, port, ip, args.port))
        if not found:
            print("no relay answered QMXR? on UDP %d" % args.discovery_port)
            sys.exit(0 if found else 1)
    if args.probe:
        sys.exit(probe(args))
    b = Bridge(args)

    def stop(*_):
        b.running = False
        signal.signal(signal.SIGTERM, stop)
        signal.signal(signal.SIGINT, stop)
        signal.signal(signal.SIGHUP, stop)
        b.run()
        log.info("stopped")

    if __name__ == "__main__":
        main()

```

11. Application listings (install, uninstall, readme, test tools)

Supporting files in tools/mac-serial-bridge, listed in full.

tools/mac-serial-bridge/install-mac.sh (new)

```
#!/bin/bash
# Install the QMX serial bridge as a login item (LaunchAgent) on this Mac.
# bash install-mac.sh # find the NAS by asking QMXR? (falls back to 10.0.0.137)
# bash install-mac.sh 192.168.1.20 # fixed NAS address, no search
# EXTRA="--keep-connected" bash install-mac.sh
set -euo pipefail
HOST="${1:-}"
FALLBACK="${FALLBACK:-10.0.0.137}"
LABEL="com.qmx.serial-bridge"
SRC="$(cd "$(dirname "$0")" && pwd)"
DEST="$HOME/Library/Application Support/QMX-Serial-Bridge"
PLIST="$HOME/Library/LaunchAgents/$LABEL.plist"
LOG="$HOME/Library/Logs/qmx-serial-bridge.log"
PY="/usr/bin/python3"

if ! xcode-select -p >/dev/null 2>&1; then
    echo "macOS's Python 3 needs the Xcode command line tools: run 'xcode-select --install' first."
    exit 1
fi

mkdir -p "$DEST" "$HOME/Library/LaunchAgents" "$HOME/Library/Logs"
cp "$SRC/qmx_serial_bridge.py" "$DEST/"

ARGS="<string>$PY</string><string>$DEST/qmx_serial_bridge.py</string>"
if [ -n "$HOST" ]; then
    ARGS="$ARGS<string>--host</string><string>$HOST</string>"; WHERE="NAS $HOST (fixed)"; PROBE="--host $H
    < OST"
else
    ARGS="$ARGS<string>--fallback-host</string><string>$FALLBACK</string>"
    WHERE="NAS found by QMXR? search (fallback $FALLBACK)"; PROBE="--fallback-host $FALLBACK"
fi
for a in ${EXTRA:-}; do ARGS="$ARGS<string>$a</string>"; done

cat > "$PLIST" <<EOF
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key><string>$LABEL</string>
  <key>ProgramArguments</key><array>$ARGS</array>
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
  <key>ThrottleInterval</key><integer>5</integer>
  <key>StandardOutPath</key><string>$LOG</string>
  <key>StandardErrorPath</key><string>$LOG</string>
</dict>
</plist>
EOF

launchctl bootout "gui/$(id -u)/$LABEL" 2>/dev/null || true
launchctl bootstrap "gui/$(id -u)" "$PLIST"
sleep 1
echo "Installed. Serial port: $HOME/cu.QMX-NAS ($WHERE, relay port 7356)"
echo "Log: $LOG"
tail -n 3 "$LOG" 2>/dev/null || true
echo
echo "Checking the relay:"
"$PY" "$DEST/qmx_serial_bridge.py" $PROBE --probe || true
```

tools/mac-serial-bridge/uninstall-mac.sh (new)

```
#!/bin/bash
# Remove the QMX serial bridge login item.
LABEL="com.qmx.serial-bridge"
launchctl bootout "gui/$(id -u)/$LABEL" 2>/dev/null || true
rm -f "$HOME/Library/LaunchAgents/$LABEL.plist"
rm -rf "$HOME/Library/Application Support/QMX-Serial-Bridge"
rm -f "$HOME/cu.QMX-NAS"
echo "Removed. (The log stays at ~/Library/Logs/qmx-serial-bridge.log)"
```

tools/mac-serial-bridge/README.md (new)

QMX on the NAS as a serial port on the Mac

`qmx\_serial\_bridge.py` makes the QMX that's plugged into the Synology show up on a Mac as a serial port, so programs that expect a USB-connected QMX (WSJT-X, JTDX, fldigi, Hamlib/rigctld, loggers, `screen`) can use it unchanged.

```

...
QMX —USB→ NAS (qmx_nas_relay.py) —TCP 7356 (QMX serial port 3)→
...   qmx_serial_bridge.py on the Mac → ~/cu.QMX-NAS ← WSJT-X, rigctld ...

```

It uses the relay's existing **port 3** service, so nothing changes on the NAS or the Tab5. It finds the NAS the same way the Tab5's **Find NAS** does (see below), so no address needs to be typed in. The Tab5 keeps its own connection, and both can control the radio at the same time.

Setup

1. On the radio: **System config** → GPS & Ser. ports → USB serial ports = 3.
2. On the Mac (needs only macOS's own Python 3):

```

...
cd tools/mac-serial-bridge
bash install-mac.sh          # find the NAS; or: bash install-mac.sh <NAS IP>
...

```

This installs it as a login item that starts automatically and restarts if it stops. At the end it runs a read-only check (`ID;` `FA;` `MD;`) and should print `PROBE PASSED`.

3. In your program, choose the **QMX** (or Kenwood TS-480) rig and type this serial port path: `/Users/<you>/cu.QMX-NAS`. Any baud rate works. For PTT, use **CAT** (DTR/RTS lines don't go over the network).

Run it by hand instead of installing it:

```

`python3 qmx_serial_bridge.py -v` (-v` logs every byte).
`python3 qmx_serial_bridge.py --discover` lists the relays that answer.
`bash uninstall-mac.sh` removes the login item.

```

Finding the NAS

Unless `--host` is given, the bridge broadcasts `QMXR?` on UDP 7355 (three times over 1.5 s, like the Tab5) and the relay answers straight back with `QMXR! 10.0.0.137 7355`. The bridge takes the **address** from that answer. The **port** in the answer is the relay's Tab5 service (QMXR/1, 7355); the serial port uses the relay's port 3 service on the same NAS, **TCP 7356** (`--port`), because 7355 carries the Tab5's framed I/Q stream, not raw serial.

- * It searches when it starts, and again whenever the remembered address stops answering, so a NAS that gets a new address is followed automatically.
- * If several relays answer, the first is used and the others are logged.
- * The installer adds `--fallback-host 10.0.0.137`, used only when no relay answers (for example if broadcasts are blocked). `bash install-mac.sh <IP>` uses a fixed address and never searches.
- * The NAS and the Mac must be on the same network; if DSM's firewall is on, allow UDP 7355 as well as TCP 7356.

How it behaves

- * It connects to the relay only when a program sends something, and releases the radio's port 3 after 120 s of silence (`--idle N`; `--keep-connected` to hold it all the time). A program that polls the radio, like WSJT-X, keeps it connected while it runs.
- * If the QMX is off or the NAS can't be reached, commands get no reply, just as if a USB radio were switched off, and the bridge retries on the next command. If the radio isn't set to 3 serial ports, the log says so.
- * Echo is always kept off on the port, so the radio's replies can never loop back into it.
- * Log: `~/Library/Logs/qmx-serial-bridge.log`.
- * If macOS blocks local-network access for python3 (System Settings → Privacy & Security → Local Network), both the search and the connection fail; the log points there when the NAS is unreachable.

Limits

- * **Serial (CAT) only, no audio.** For digital modes you still need the radio's audio on the Mac by some other route.
- * The port is a pseudo-terminal at a path you choose, not a USB device in macOS's device list. Programs whose port list can't be edited won't list it. (Making macOS see a real USB serial device would need a DriverKit driver)

signed with entitlements from Apple.) Hamlib-based programs (WSJT-X, JTDX, fldigi, rigctld) accept a typed path.
 * One PC at a time on port 3. A second Mac connecting takes over from the first.

tools/mac-serial-bridge/test/fake_port3_relay.py (new)

```
#!/usr/bin/env python3
# Bench test: stand-in for qmx_nas_relay.py port 3 (TCP 17356) with a tiny CAT emulator.
# Usage: python3 fake_port3_relay.py [ok|refuse] (refuse = act like a radio without port 3)
import socket, sys, threading
mode = sys.argv[1] if len(sys.argv) > 1 else "ok"
srv = socket.socket(); srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("127.0.0.1", 17356)); srv.listen(2)
freq = [14074000]
def handle(c):
    if mode == "refuse": c.close(); print("refused", flush=True); return
    print("client", flush=True); buf = b""
    while True:
        d = c.recv(4096)
        if not d: print("client gone", flush=True); return
        print("got", d, flush=True); buf += d
        while b";" in buf:
            cmd, buf = buf.split(b";", 1)
            if cmd == b"FA": c.sendall(b"FA%011d;" % freq[0])
            elif cmd.startswith(b"FA"): freq[0] = int(cmd[2:])
            elif cmd == b"ID": c.sendall(b"ID020;")
            elif cmd == b"MD": c.sendall(b"MD3;")
    while True:
        c, _ = srv.accept(); threading.Thread(target=handle, args=(c,), daemon=True).start()
```

tools/mac-serial-bridge/test/port_client.py (new)

```
#!/usr/bin/env python3
# Bench test: open the bridge serial port like an application and send CAT commands.
# Usage: python3 port_client.py <port path> "FA;" ["ID;" ...] (ECHO=1 turns echo on first)
import os, sys, termios, tty, time, select
path = os.path.expanduser(sys.argv[1]); cmds = sys.argv[2:]
fd = os.open(path, os.O_RDWR | os.O_NOCTTY)
if os.environ.get("ECHO"): # simulate a sloppy app that turns echo on
    a = termios.tcgetattr(fd); a[3] |= termios.ECHO; termios.tcsetattr(fd, termios.TCSANOW, a)
for c in cmds:
    os.write(fd, c.encode()); r = b""; end = time.time() + 5
    while time.time() < end and not r.endswith(b";"):
        if select.select([fd], [], [], 0.2)[0]: r += os.read(fd, 256)
    print(c, "->", r.decode() or "(none)")
os.close(fd)
```